Universidad Autónoma del Estado de México
Programa Delfín 2026
Research Line: Theory of Computation

Planar Graph Coloring via Topological Peeling and CSP Backtracking

Mathematical and algorithmic documentation
of a computational visualization of
the Four Color Theorem

Author: Yosef Ali Jiménez Muñoz
Student ID: 492410631
Home Institution: Universidad Tecnológica de Altamira
Academic Advisor: Dr. José Raymundo Marcial Romero
Program: Programa Delfín, Summer 2026
Date: June 2026

Abstract

We document the mathematical and algorithmic design of a computational visualization system for the Four Color Theorem, implemented in Python with the libraries `NetworkX`, `NumPy`, `SciPy`, and `Matplotlib`. The system operates on arbitrary planar graphs and executes a transformation pipeline that includes: planarity verification via the Boyer-Myrvold algorithm [3]; straight-line drawing via the Tutte embedding [21]; topological layer decomposition via outer-face peeling; inverse-order greedy coloring; analysis of Kempe chains [12]; and CSP backtracking with MRV and forward checking when the greedy fails to produce four colors. The Errera [6] and Kittell [13] graphs are used as canonical test cases, concretely demonstrating the failure mechanism of Kempe's algorithm via tangled chains. The document includes complete mathematical foundations for each step, precise numerical values computed on the Errera graph, and situates the work historically from Kempe's incorrect proof to Gonthier's formal verification in Coq [9].

Contents

1 Introduction

The Four Color Theorem states that every planar graph can be colored with at most four colors so that no two adjacent vertices share a color. Formulated as a conjecture in 1852 by Francis Guthrie while coloring a map of English counties, it resisted proof for over a century [22].

The first proof was proposed by Alfred Kempe in 1879 [12], but Percy Heawood identified a fundamental error in 1890 [11]. In 1921, Alfred Errera constructed a concrete planar graph exhibiting the exact failure point of Kempe's algorithm [6]. A correct proof did not arrive until 1976, when Appel and Haken computationally verified that a finite set of configurations covers all possible cases [2]. Robertson, Sanders, Seymour, and Thomas simplified the proof to 633 configurations in 1997 [17], and in 2005 Gonthier produced the first mechanically checkable formal verification in the Coq proof assistant [9].

This document describes a visualization system that implements the algorithmic steps of the coloring process, from the representation of a graph as ordered pairs to the guarantee of four colors via CSP backtracking. The goal is not to prove the theorem (whose proof requires the global discharging argument) but to make visible the mathematical mechanisms underlying the problem: Euler's formula, triangulation, barycentric embedding, topological peeling, greedy coloring, Kempe chain failure, and constraint-based search.

2 Graph Representation

2.1 Formal Definition

Definition 2.1 (Simple undirected graph)**.** A graph $G = (V, E)$ consists of a finite set of vertices V and a set of edges $E \subseteq \binom{V}{2}$, where $\binom{V}{2}$ denotes all two-element subsets of V. Each edge $e = \{u, v\} \in E$ is an unordered pair with $u \neq v$.

The **degree** of a vertex v is $\deg(v) = |\{u \in V : \{u, v\} \in E\}|$. The following identity holds because each edge contributes 1 to the degree of each of its two endpoints:

$$\sum_{v \in V} \deg(v) = 2|E|. \tag{1}$$

2.2 Input Format

The system reads a `grafo.json` file with the structure:

```
{
  "vertices": {"0": [x0, y0], "1": [x1, y1], ...},
  "aristas":  [[0, 1], [1, 2], ...]
}
```

Listing 1: Input file structure

The coordinates (x_i, y_i) are used initially for geometric planarity checks, but the system then computes a canonical embedding via the Tutte algorithm (Section 5). Conversion from the edge list to the `NetworkX` adjacency structure runs in $O(|E|)$.

2.3 The Errera Graph

The Errera graph [6] has $|V| = 17$ vertices and $|E| = 45$ edges. Its degree sequence is twelve vertices of degree 5 and five of degree 6. Verification: $12 \cdot 5 + 5 \cdot 6 = 90 = 2 \cdot 45$, consistent with (1). Its vertex connectivity is $\kappa(G) = 5$ [6].

The graph has a concentric layered structure:

Layer	Vertices	Role
Bottom apex	$\{0\}$	Degree-5 vertex
Ring 1	$\{1, 2, 3, 4, 5\}$	Inner pentagon
Ring 2	$\{6, 7, 8, 9, 10\}$	Middle pentagon
Ring 3	$\{11, 12, 13, 14, 15\}$	Outer pentagon
Top apex	$\{16\}$	Degree-5 vertex

This structure corresponds to two stacked pentagonal antiprisms with a 5-fold dihedral symmetry group D_5 [6].

3 Planarity Verification

3.1 Definition and Fundamental Theorems

Definition 3.1 (Planar graph)**.** A graph G is planar if there exists an embedding $\varphi : G \to \mathbb{R}^2$ such that each vertex maps to a distinct point, each edge maps to a Jordan curve between its endpoints, and curves of distinct edges do not intersect except possibly at shared endpoints.

Theorem 3.2 (Fáry, 1948 [8])**.** *Every simple planar graph admits a straight-line embedding: a drawing where every edge is a line segment.*

Fáry's theorem justifies working with coordinates and segments without loss of generality.

Theorem 3.3 (Kuratowski, 1930 [14])**.** *A graph is planar if and only if it contains no subgraph homeomorphic to K_5 or $K_{3,3}$.*

3.2 Boyer-Myrvold Algorithm

The system calls `nx.check_planarity(G)`, which internally runs the Boyer-Myrvold algorithm [3].

Theorem 3.4 (Boyer-Myrvold, 1999 [3]). *The Boyer-Myrvold algorithm determines planarity in $O(|V|)$ time and, when the graph is planar, constructs the combinatorial embedding.*

The algorithm operates on the combinatorial representation, not on coordinates. For each vertex v it stores the cyclic order $(\sigma_v(e_1), \sigma_v(e_2), \ldots)$ of incident edges, where σ_v is a cyclic permutation. This structure—the **combinatorial embedding**—is sufficient to determine faces without coordinates.

Geometric cross-checking with `Shapely` is an additional validation step applied to the input JSON. The intersection test between two segments $\overline{p_1p_2}$ and $\overline{p_3p_4}$ is based on the sign of the cross product:

$$(p_2 - p_1) \times (p_3 - p_1) \quad \text{and} \quad (p_2 - p_1) \times (p_4 - p_1). \tag{2}$$

The segments intersect in their interiors if and only if the signs differ in both reciprocal checks [20].

4 Euler's Formula and Maximality

4.1 Euler's Formula

Theorem 4.1 (Euler, 1752 [7]). *For every connected planar graph, letting F denote the number of faces (regions, including the unbounded exterior face):*

$$V - E + F = 2. \tag{3}$$

Proof sketch. By induction on $|E|$. A tree with n vertices has $n - 1$ edges and one face, giving $n - (n - 1) + 1 = 2$. Adding an edge between two components: $\Delta E = 1$, $\Delta F = 0$, so $V - E + F$ is preserved. Adding an edge within a component: $\Delta E = 1$, $\Delta F = 1$, also preserved. □

For the Errera graph: $17 - 45 + 30 = 2$ ✓, with 30 triangular faces.

4.2 Density Bound

Theorem 4.2. *For every simple planar graph with $|V| \geq 3$:*

$$|E| \leq 3|V| - 6. \tag{4}$$

Proof. Every face has at least 3 edges on its boundary, and each edge borders exactly 2 faces, so $3F \leq 2|E|$, i.e., $F \leq \frac{2|E|}{3}$. Substituting into (3):

$$|V| - |E| + \frac{2|E|}{3} \geq 2 \implies |V| - \frac{|E|}{3} \geq 2 \implies |E| \leq 3|V| - 6. \qquad \square$$

4.3 Maximal Planar Graphs

Definition 4.3 (Maximal planar graph)**.** A planar graph G is maximal if for every $\{u, v\} \notin E$, the graph $G' = (V, E \cup \{\{u, v\}\})$ is non-planar.

Theorem 4.4. *A planar graph with $|V| \geq 3$ is maximal if and only if every face (including the outer face) is a triangle.*

When the outer face is also a triangle, equality holds in (4): $|E| = 3|V| - 6$. In the general case where the outer face has h vertices:

$$|E| = 3|V| - 6 - (h - 3). \tag{5}$$

For the Errera graph: $|E| = 45 = 3(17) - 6 = 45$ ✓, with $h = 3$. The graph is already maximal, so the maximalization step adds zero edges.

4.4 Existence of a Low-Degree Vertex

Theorem 4.5. *In every planar graph there exists at least one vertex v with* $\deg(v) \leq 5$.

Proof. Suppose $\deg(v) \geq 6$ for all v. Then:

$$2|E| = \sum_{v} \deg(v) \geq 6|V| \implies |E| \geq 3|V|.$$

But Theorem 4.2 requires $|E| \leq 3|V| - 6 < 3|V|$. Contradiction. $\square$

This result guarantees that the inverse greedy coloring always finds a free color among six (Section 7).

4.5 Constrained Maximalization

The input graph may not be maximal. The system adds edges without crossing any existing ones:

Algorithm 1 Constrained maximalization

Require: Planar graph $G = (V, E)$ with positions $\mathbf{p} : V \to \mathbb{R}^2$
Ensure: Maximal planar graph G' with $E \subseteq E'$
1: $\mathcal{C} \leftarrow \binom{V}{2} \setminus E$
2: Sort $\mathcal{C}$ by $\deg(u) + \deg(v)$ ascending
3: **for** each $\{u, v\} \in \mathcal{C}$ **do**
4: **if** $\mathbf{p}(u)\mathbf{p}(v)$ crosses no edge of E **then**
5: $E \leftarrow E \cup \{\{u, v\}\}$
6: **end if**
7: **end for**
8: **return** $G' = (V, E)$

Sorting by minimum degree first connects the most isolated vertices before the densely connected ones, producing a more balanced graph.

5 Tutte Embedding

5.1 Tutte's Theorem

Theorem 5.1 (Tutte, 1963 [21])**.** *Let G be a 3-connected planar graph with outer face $F_0 = (v_1, \ldots, v_k)$ fixed on a convex polygon in $\mathbb{R}^2$. If every interior vertex satisfies the **barycentric condition**:*

$$\mathbf{p}(v) = \frac{1}{\deg(v)} \sum_{u \sim v} \mathbf{p}(u), \tag{6}$$

then the resulting embedding is a crossing-free straight-line drawing.

5.2 The Linear System

Rearranging (6) for each interior vertex v_i and separating interior neighbors from boundary neighbors:

$$\deg(v_i)\,\mathbf{p}(v_i) - \sum_{\substack{u \sim v_i \\ u \in \text{interior}}} \mathbf{p}(u) = \sum_{\substack{u \sim v_i \\ u \in F_0}} \mathbf{p}(u). \tag{7}$$

In matrix form, for the x and y coordinates separately:

$$L_{\text{int}}\,\mathbf{x} = \mathbf{b}_x, \qquad L_{\text{int}}\,\mathbf{y} = \mathbf{b}_y, \tag{8}$$

where L_{int} is the **interior Laplacian submatrix**:

$$(L_{\text{int}})_{ij} = \begin{cases} \deg(v_i) & i = j, \\ -1 & i \neq j \text{ and } v_i \sim v_j, \\ 0 & \text{otherwise.} \end{cases} \tag{9}$$

The right-hand side vector $\mathbf{b}_x$ has at position i the sum of the x-coordinates of the boundary neighbors of v_i.

5.3 Properties of the Laplacian

Theorem 5.2. *The matrix L_{int} defined in (9) is symmetric positive definite (SPD).*

Proof. L_{int} is the submatrix of the graph Laplacian induced by the interior nodes. The full graph Laplacian of a connected graph has a single zero eigenvalue with eigenvector proportional to the all-ones vector. Fixing the boundary nodes (removing the corresponding rows and columns) eliminates this zero eigenvalue, making the submatrix invertible. Positive definiteness follows from spectral graph theory [5]: all eigenvalues of L_{int} are strictly positive. □

Since L_{int} is SPD, system (8) has a unique solution, computed by `numpy.linalg.solve` via Gaussian elimination with partial pivoting in $O(m^3)$, where m is the number of interior nodes.

5.4 Application to the Errera Graph

The outer face $\{0, 1, 2\}$ is fixed on an equilateral triangle of radius 8:

$$\begin{aligned}\mathbf{p}(0) &= (0,\ 8),\\ \mathbf{p}(1) &= (-6.928,\ -4),\\ \mathbf{p}(2) &= (6.928,\ -4).\end{aligned}$$

The 14 interior nodes $\{3, 4, \ldots, 16\}$ generate a 14×14 system with condition number $\kappa(L_{\text{int}}) \approx 18.25$, indicating a well-conditioned system. The upper-left 4×4 block of L_{int} is:

$$\begin{pmatrix} 5 & -1 & 0 & 0 \\ -1 & 5 & -1 & 0 \\ 0 & -1 & 5 & -1 \\ 0 & 0 & -1 & 6 \end{pmatrix} \tag{10}$$

Selected positions after solving the system:

$$\begin{aligned}\mathbf{p}(6) &= (-1.7108,\ -1.0880),\\ \mathbf{p}(11) &= (-0.5338,\ -0.6087),\\ \mathbf{p}(16) &= (0.0,\ -0.7563).\end{aligned}$$

The outer face is selected by maximizing the sum of eccentricities. The **eccentricity** of a vertex is:

$$\text{ecc}(v) = \max_{u \in V} d(v, u), \tag{11}$$

where $d(v, u)$ is the shortest-path distance, computed via BFS in $O(|V| + |E|)$. In the Errera graph: $\mathrm{ecc}(0) = \mathrm{ecc}(16) = 4$, $\mathrm{ecc}(v) = 3$ for all others, giving $\sum_{v\in\{0,1,2\}} \mathrm{ecc}(v) = 10$ as the maximum over all faces.

6 Topological Peeling

6.1 Face Traversal: $\sigma \circ \alpha^{-1}$

Given the combinatorial embedding, faces are computed via dart traversal. A **dart** is a directed edge $(v \to w)$. Two operations are defined:

$$\alpha(v \to w) = (w \to v) \qquad \text{(dart reversal)}, \tag{12}$$

$$\sigma_w(v \to w) = \text{next dart clockwise around } w. \tag{13}$$

The composition $\sigma \circ \alpha^{-1}$ traverses the face adjacent to each dart: starting from $(v_0 \to v_1)$, the sequence

$$(v_0 \to v_1),\ (\sigma \circ \alpha^{-1})(v_0 \to v_1),\ \ldots$$

eventually closes at $(v_0 \to v_1)$, and the visited vertices form the face [16]. This is implemented by `emb.traverse_face(v, w)` in `NetworkX`.

6.2 Peeling Algorithm

Algorithm 2 Topological peeling

Require: Maximal planar graph G
Ensure: Elimination order $\pi = (v_1, v_2, \ldots, v_n)$

$G' \leftarrow G$, $\pi \leftarrow ()$
while $V(G') \neq \emptyset$ **do**
  Compute planar embedding of G' (Boyer-Myrvold)
  $\mathcal{F} \leftarrow$ all faces via dart traversal
  $F^* \leftarrow \arg\max_{F\in\mathcal{F}} \sum_{v\in F} \mathrm{ecc}_{G'}(v)$
  $v^* \leftarrow \arg\min_{v\in F^*} \deg_{G'}(v)$
  $\pi \leftarrow \pi \cdot (v^*)$
  $G' \leftarrow G' \setminus \{v^*\}$
end while
return π

6.3 Results on the Errera Graph

The complete peeling order and degrees at the moment of elimination are:

Step	Node	Outer face	Deg.	Step	Node	Outer face	Deg.
1	0	$\{0,1,2\}$	5	10	9	$\{9,10,11,12,13,14\}$	3
2	1	$\{1,2,3,4,5\}$	4	11	10	$\{10,11,12,13,14,15\}$	2
3	2	$\{2,3,4,5,6,7\}$	3	12	11	$\{11,12,13,14,15\}$	3
4	3	$\{3,4,5,6,7,8\}$	3	13	12	$\{12,13,14,15,16\}$	2
5	4	$\{4,5,6,7,8,9\}$	3	14	13	$\{13,14,15,16\}$	2
6	5	$\{5,6,7,8,9,10\}$	2	15	14	$\{14,15,16\}$	2
7	6	$\{6,7,8,9,10\}$	4	16	15	$\{15,16\}$	1
8	7	$\{7,8,9,10,11,12\}$	3	17	16	$\{16\}$	0
9	8	$\{8,9,10,11,12,13\}$	3				

The maximum degree at elimination is 5 (step 1), consistent with Theorem 4.5. The **degeneracy** of the graph [15] is $d(G) = \max_i \deg_{G_i}(v_i) = 5$.

7 Inverse Greedy Coloring

7.1 The Algorithm

Given the peeling order $\pi = (v_1, \ldots, v_n)$, coloring proceeds in reverse order $v_n, v_{n-1}, \ldots, v_1$:

Algorithm 3 Inverse greedy coloring

Require: Graph G, peeling order $\pi = (v_1, \ldots, v_n)$
Ensure: Coloring $c : V \to \mathbb{N}$
1: $c \leftarrow \{\}$
2: **for** $i = n, n-1, \ldots, 1$ **do**
3: $\quad \mathcal{P}(v_i) \leftarrow \{c(u) : u \in N(v_i),\ u \in \mathrm{dom}(c)\}$
4: $\quad c(v_i) \leftarrow \min(\mathbb{N} \setminus \mathcal{P}(v_i))$
5: **end for**
6: **return** c

7.2 Six-Color Guarantee

Theorem 7.1 (Six Color Theorem [11])**.** *Every planar graph is 6-colorable.*

Proof. By induction on $|V|$. By Theorem 4.5, there exists v with $\deg(v) \leq 5$. By the inductive hypothesis, $G \setminus \{v\}$ is 6-colorable. Reinserting v, it has at most 5 neighbors using at most 5 distinct colors. Of the 6 available colors, at least one is free. □

The inverse greedy coloring implements this argument constructively: the reverse peeling order guarantees that when v_i is colored, its already-colored neighbors are exactly those it had when removed during peeling, and at that moment $\deg(v_i) \leq 5$. □

7.3 Results on the Errera Graph

The inverse greedy coloring produces a valid 4-coloring of the Errera graph directly. The complete assignment is:

Node	Forbidden	Color	Node	Forbidden	Color	Node	Forbidden	Color
16	$\emptyset$	0	9	$\{0,1,2\}$	3	2	$\{0,2,3\}$	1
15	$\{0\}$	1	8	$\{1,2,3\}$	0	1	$\{1,2,3\}$	0
14	$\{0,1\}$	2	7	$\{0,1,2\}$	3	0	$\{0,1,2\}$	3
13	$\{0,2\}$	1	6	$\{0,2,3\}$	1			
12	$\{0,1\}$	2	5	$\{0,1\}$	2			
11	$\{0,1,2\}$	3	4	$\{0,2,3\}$	1			
10	$\{1,3\}$	0	3	$\{0,1,3\}$	2			

The resulting coloring uses $|\{c(v) : v \in V\}| = 4$ colors and is verified valid: $\forall\{u,v\} \in E : c(u) \neq c(v)$.

8 Kempe Chains and the Errera Counterexample

8.1 Kempe Chains

Definition 8.1 (Kempe chain [12])**.** Given a partial coloring c and two colors c_i, c_j, the (c_i, c_j)-Kempe chain from v is the connected component of v in the subgraph induced by vertices colored c_i or c_j:

$$K_{ij}(v) = \text{connected component of } v \text{ in } G[\{u : c(u) \in \{c_i, c_j\}\}]. \tag{14}$$

A **Kempe swap** exchanges $c_i \leftrightarrow c_j$ on all nodes of $K_{ij}(v)$. The resulting coloring remains valid because $K_{ij}(v)$ is a connected component: no edges cross from inside $K_{ij}(v)$ to external nodes with color c_i or c_j [12].

8.2 Kempe's Error and the Errera Graph

Kempe assumed that whenever the current outer face uses 4 colors and the next vertex to insert has all 4 forbidden, it is always possible to reduce the outer face to 3 colors via a Kempe swap. This assumption is false, as Heawood showed in 1890 [11].

The Errera graph was constructed to exhibit this impossibility concretely [6]. The system analyzes all candidate chains (c_i, c_j) at each conflicting step, classifying each as:

- **SUCCESS**: the swap reduces the outer face to ≤ 3 colors.

- **TANGLED**: the chain connects two outer-face nodes of colors c_i and c_j, making the swap useless.
- **NO_HELP**: valid swap but the outer face still uses 4 colors.

Definition 8.2 (Tangled Kempe chain)**.** A (c_i, c_j)-chain $K_{ij}(v)$ from v (with $c(v) = c_i$) is *tangled* if it contains a node w on the outer face with $c(w) = c_j$. The swap exchanges $c_i \to c_j$ at v and $c_j \to c_i$ at w, leaving both colors present on the outer face.

At step 14 of Act 2 (insertion of node 3) in the Errera graph, all candidate chains on the outer face are tangled. The system checks the $\binom{4}{2} = 6$ color pairs and all possible chains from outer-face nodes, finding that none reduces the outer face from 4 to 3 colors. This is the concrete instance of the phenomenon documented by Errera [6].

The Kittell graph [13], with 23 vertices and $\chi(G) = 4$, exhibits the same phenomenon at multiple steps during the Act 2 coloring.

9 CSP Backtracking with Four-Color Guarantee

9.1 CSP Formulation

When the greedy coloring with Kempe swaps produces more than 4 colors, the system activates backtracking over the CSP formulation:

Definition 9.1 (Constraint Satisfaction Problem, CSP)**.** The 4-coloring problem for $G = (V, E)$ is formulated as a CSP with:

- Variables: $X = \{x_v : v \in V\}$
- Domains: $D(x_v) = \{0, 1, 2, 3\}$ for all v
- Constraints: $x_u \neq x_v$ for all $\{u, v\} \in E$

The search space is $|D|^{|V|} = 4^{17} \approx 1.7 \times 10^{10}$ for the Errera graph. Pruning heuristics reduce the number of recursive calls to 22 in practice.

9.2 MRV Heuristic

Definition 9.2 (Minimum Remaining Values, MRV [18])**.** The MRV heuristic selects at each step the unassigned variable with the smallest domain:

$$v^* = \text{argmin}_{v \notin \text{dom}(c)} \, |\,\text{dom}(x_v)|. \tag{15}$$

Ties are broken by choosing the vertex of highest degree (*degree heuristic*).

MRV implements the "fail-first" principle: by choosing the most constrained variable, failures are detected early and more of the search tree is pruned [10].

9.3 Forward Checking

Definition 9.3 (Forward Checking [10])**.** Upon assigning color c to vertex v, forward checking removes c from the domain of each uncolored neighbor $u \in N(v) \setminus \mathrm{dom}(c)$:

$$\mathrm{dom}(x_u) \leftarrow \mathrm{dom}(x_u) \setminus \{c\} \quad \forall u \in N(v),\ u \notin \mathrm{dom}(c). \tag{16}$$

If any $\mathrm{dom}(x_u)$ becomes empty, the current assignment is inconsistent and the algorithm backtracks immediately.

9.4 Complete Algorithm

Algorithm 4 Backtracking with MRV and Forward Checking

Require: Graph $G = (V, E)$
Ensure: Coloring $c : V \to \{0, 1, 2, 3\}$ or FAILURE

$c \leftarrow \{\}$; $\mathrm{dom}(x_v) \leftarrow \{0, 1, 2, 3\}$ for all v
function BACKTRACK
  $v^* \leftarrow \mathrm{argmin}_{v \notin \mathrm{dom}(c)} |\mathrm{dom}(x_v)|$ (MRV)
  if $v^* = \emptyset$ **then return DONE**
  end if
  for $\ell \in \mathrm{dom}(x_{v^*})$ in ascending order **do**
    if $\ell \notin \{c(u) : u \in N(v^*) \cap \mathrm{dom}(c)\}$ **then**
      pruned $\leftarrow \emptyset$
      for $u \in N(v^*) \setminus \mathrm{dom}(c)$ **do** (Forward Check)
        if $\ell \in \mathrm{dom}(x_u)$ **then**
          $\mathrm{dom}(x_u) \leftarrow \mathrm{dom}(x_u) \setminus \{\ell\}$
          pruned $\leftarrow$ pruned $\cup \{u\}$
          if $\mathrm{dom}(x_u) = \emptyset$ **then**
            Restore pruned; **continue** to next ℓ
          end if
        end if
      end for
      $c(v^*) \leftarrow \ell$
      if BACKTRACK = **DONE then return DONE**
      end if
      $c(v^*) \leftarrow$ undefined; restore domains of pruned
    end if
  end for
  return FAILURE
end function

9.5 Search Trace Visualization

The system records each event of the search tree for frame-by-frame visualization:

Event	Frame	Meaning
`ASSIGN`	Color fixed, neighbors pruned	Successful assignment; node shown with thick white border
`FAIL`	Node in dark red	Domain wipeout in some neighbor; must backtrack
`UNASSIGN`	Node decolored	Undoes assignment and restores domains
`DONE`	All nodes colored	Solution found

9.6 Results on the Errera Graph

Parameter	Value
Recursive calls	22
`ASSIGN` events	21
`FAIL` / `UNASSIGN` events	4 each
Colors used	4
Valid coloring	Yes
Full search space	$4^{17} \approx 1.7 \times 10^{10}$

The coloring found by backtracking is: {6:0, 7:1, 1:2, 2:0, 8:2, 0:1, 3:3, 5:3, 9:0, 4:2, 10:1, 11:3, 12:2, 1
differing from the greedy coloring in 15 of 17 nodes.

10 The Four Color Theorem: Historical Context

10.1 Timeline

1852 Francis Guthrie formulates the conjecture while coloring maps [22].

1879 Alfred Kempe proposes a proof based on chain interchange [12].

1890 Percy Heawood identifies the flaw in Kempe's proof and proves the Five Color Theorem [11].

1921 Alfred Errera constructs a concrete planar graph exhibiting the failure point of Kempe's algorithm [6].

1935 I. Kittell constructs a second counterexample to Kempe's algorithm [13].

1976 Appel and Haken prove the Four Color Theorem by computationally verifying 1936 reducible configurations [2].

1997 Robertson, Sanders, Seymour, and Thomas simplify the proof to 633 configurations with an independent verifier [17].

2005 Georges Gonthier produces the first mechanically verified formal proof in the Coq proof assistant [9].

10.2 Gonthier's Formal Verification

Gonthier's contribution [9] shifts the object of trust: rather than trusting a 500,000-line verification program, one trusts only the Coq kernel (approximately 3,000 lines), whose correctness can be established independently.

The formalism uses **combinatorial hypermaps**: an algebraic representation of a planar embedding based on two permutations over a set of darts. If H denotes the dart set and $\sigma, \alpha : H \to H$ are permutations with $\alpha^2 = \mathrm{id}$, then (H, σ, α) is a hypermap representing the embedding. Faces are orbits of $\sigma \circ \alpha$, vertices are orbits of σ, and edges are orbits of α [9].

This is isomorphic to the combinatorial embedding used by `NetworkX` (implemented as a dict-of-dicts storing the cyclic neighbor order), though Gonthier's presentation is more formal and amenable to mechanical verification.

10.3 What the Code Implements and What It Does Not

The implemented system constructively demonstrates that the Errera and Kittell graphs are 4-colorable. It does not prove the Four Color Theorem for arbitrary planar graphs, which requires:

1. The **discharging method**: assign charge $\mu(v) = 6 - \deg(v)$ to each vertex (the total sum is 12 by Euler) and redistribute it via discharge rules that yield a contradiction if the graph contains no reducible configuration.
2. **Reducibility**: for each of the 633 configurations, show that any coloring of the outer ring can be extended to the interior (possibly with Kempe swaps).
3. **Unavoidability**: show that the configuration set is unavoidable, i.e., every planar graph contains at least one [17].

11 Computational Complexity

Table 1 summarizes the complexity of each pipeline stage.

Table 1: Complexity of each pipeline stage

Stage	Algorithm	Complexity	Reference
Graph loading	Adjacency list insertion	$O(\|E\|)$	—
Planarity check	Boyer-Myrvold	$O(\|V\|)$	[3]
Maximalization	Brute force + Shapely	$O(\|V\|^2\|E\|)$	—
Tutte embedding	Gaussian elim. ($m \times m$)	$O(\|V\|^3)$	[21]
BFS (eccentricity)	BFS from each vertex	$O(\|V\|(\|V\| + \|E\|))$	—
Face traversal	Dart traversal	$O(\|F\|)$	[16]
Full peeling	Peeling × Boyer-Myrvold	$O(\|V\|^2)$	—
Greedy coloring	Greedy over adjacency	$O(\|V\| + \|E\|)$	—
CSP backtracking (worst)	MRV + FC	$O(4^{\|V\|})$	[10]
CSP backtracking (practice)	Planar structure	$O(\|V\|)$	—

The theoretical bottleneck of backtracking is exponential in the worst case, but in practice the structure of planar graphs causes MRV + FC to solve the problem in a few dozen recursive calls. The Errera graph required 22 calls instead of 4^{17}.

12 Implementation

12.1 Dependencies

Library	Version	Primary use
`networkx`	≥ 3.0	Graph structure, planar embedding, BFS
`numpy`	≥ 1.24	Tutte linear system, linear algebra
`scipy`	≥ 1.10	`ConvexHull` (legacy visual aid)
`shapely`	≥ 2.0	Segment intersection test
`matplotlib`	≥ 3.7	Visualization and GIF animation
`pillow`	≥ 9.0	GIF export

12.2 Code Structure

The main file `onion_peeling.py` organizes the pipeline into eight numbered sections, each implementing one mathematical stage:

1. **Loading**: JSON reading and graph construction.
2. **Maximalization**: Algorithm 1.
3. **Tutte embedding**: Laplacian system, Section 5.
4. **Outer face**: face traversal and eccentricity.
5. **Peeling**: Algorithm 2.
6. **Greedy coloring**: Algorithm 3.

7. **Kempe chains**: chain analysis, Section 8.
8. **Backtracking**: Algorithm 4.
9. **Animation**: three-act GIF generation.

The system produces two main outputs:

- `descomposicion_grafo.gif`: animated process on the input graph.
- `explainer.gif`: 40-frame pedagogical GIF documenting each mathematical stage.

13 Layer-by-Layer Tabü Coloring

13.1 Motivation

The greedy inverse coloring of Section 7 inserts vertices one at a time in reverse peel order. When it produces more than four colors, the system falls back to CSP backtracking (Section 9), which is guaranteed to find a four-coloring but may explore a large portion of the search space. The question motivating this section is: *can the coloring be organized so that backtracking is never needed?*

The key observation is that the topological peeling naturally groups vertices into **layers**: all vertices on the outer face at a given moment form one layer, and removing that layer exposes the next. Coloring an entire layer at once, before moving inward, provides more global information at each decision point than the node-by-node greedy approach.

13.2 Algorithm

Definition 13.1 (Peeling layers)**.** The peeling layers $L_0, L_1, \ldots, L_k$ of a planar graph G are defined iteratively: L_0 is the outer face of G; for $i > 0$, L_i is the outer face of $G \setminus (L_0 \cup \cdots \cup L_{i-1})$.

For the Errera graph the layers are:

$$L_0 = \{0, 1, 2\}, \quad L_1 = \{3, 4, 5, 6, 7, 8\}, \quad L_2 = \{9, 10, 11, 12, 13, 14\}, \quad L_3 = \{15, 16\}.$$

Definition 13.2 (Tabü constraint)**.** When coloring a vertex $v \in L_i$, the **tabü** set of v is the set of colors already assigned to its direct neighbors in previously colored layers:

$$\tau(v) = \{c(u) : u \in N(v),\ u \in L_0 \cup \cdots \cup L_{i-1}\}. \tag{17}$$

The coloring rule is identical to the greedy rule of Section 7 — assign the minimum color not in $\tau(v)$ — but the *scope* differs: here $\tau(v)$ is computed from all previously colored layers, whereas in the node-by-node greedy it includes only the previously inserted nodes in peel order. Within each layer, nodes are colored in ascending ID order.

Algorithm 5 Layer-by-layer tabü coloring

Require: Planar graph G
Ensure: Coloring $c : V \to \mathbb{N}$
$c \leftarrow \{\}, \quad i \leftarrow 0$
while $G \neq \emptyset$ **do**
  $L_i \leftarrow$ outer face of G ▷ face traversal, max ecc sum
  for $v \in L_i$ in ascending order **do**
    $\tau(v) \leftarrow \{c(u) : u \in N(v),\ u \in \mathrm{dom}(c)\}$
    $c(v) \leftarrow \min(\mathbb{N} \setminus \tau(v))$
  end for
  $G \leftarrow G \setminus L_i, \quad i \leftarrow i + 1$
end while
return c

13.3 Relationship to Greedy and to Backtracking

The tabü algorithm is strictly more informed than the node-by-node greedy: when a vertex $v \in L_i$ is colored, *all* of its neighbors in $L_0, \ldots, L_{i-1}$ are already colored, whereas in the greedy they may not be (depending on the peel order within earlier layers). This means $\tau(v)$ is at least as large as the forbidden set in the node-by-node greedy, so the tabü algorithm makes more constrained — and therefore more globally consistent — decisions.

The tabü constraint (17) is equivalent to the *initial* forward-checking propagation in the CSP backtracking: before assigning any color to v, backtracking would also eliminate the colors of all colored neighbors from v's domain. The difference is that the tabü algorithm never revises a decision, whereas backtracking undoes assignments when a domain becomes empty.

13.4 Results on the Errera Graph

Applying Algorithm 5 to the Errera graph:

Layer	Nodes	Tabü sets	Colors assign
L_0	$\{0,1,2\}$	$\emptyset$, $\{0\}$, $\{0,1\}$	0, 1, 2
L_1	$\{3,4,5,6,7,8\}$	$\{0,2\}$, $\{0,1\}$, $\{0,1,2\}$, $\{1,3\}$, $\{0,1,2\}$, $\{1,2,3\}$	1, 2, 3, 0, 3, 0
L_2	$\{9,10,11,12,13,14\}$	$\{0,1,2\}$, $\{0,2,3\}$, $\{0,1\}$, $\{0,2,3\}$, $\{0,1,3\}$, $\{0,2,3\}$	3, 1, 2, 1, 2, 1
L_3	$\{15,16\}$	$\{1,2,3\}$, $\{0,1,2\}$	0, 3

The algorithm produces a valid 4-coloring **without backtracking**. The maximum tabü set size is 3 (occurring at nodes 5, 7, 8 in L_1, and several nodes in L_2), consistent with the bound $|\tau(v)| \leq 5$ that follows from the density argument of Theorem 4.5.

13.5 Comparison with Other Methods

Method	Colors	Backtracks	Assignments	Complexity
Greedy (node-by-node)	4	0	17	$O(V+E)$
Greedy + Kempe swaps	5	0	17	$O(V(V+E))$
CSP Backtracking (MRV+FC)	4	4	21	$O(4^V)$ worst
Tabü layer-by-layer	4	0	17	$O(V+E)$

The tabü algorithm matches the result of CSP backtracking (4 colors, valid) without any backtracking, and runs in $O(V+E)$ — the same asymptotic complexity as the greedy. The key open question is whether this holds for all planar graphs or only for structured cases like Errera.

14 Bi-Layer Coloring: Failure Detection and Closure

14.1 Motivation and Overview

De Ita Luna and Marcial-Romero [1] propose the Bi-Layer algorithm, which extends the outer-face greedy approach with two key mechanisms: (1) *failure configuration detection* — recognizing local graph patterns that would require a fifth color before they are reached — and (2) the *closure operator* $T((v_i, a))$ — simulating the transitive propagation of forced colorings when color a is tentatively assigned to vertex v_i, in order to evaluate the downstream consequences of each color choice.

14.2 Failure Configurations

Definition 14.1 (Basic failure configurations [1])**.** Given a partial coloring c and tabu sets $\tau(v) = \{c(u) : u \sim v,\, u \in \mathrm{dom}(c)\}$:

- **Failed vertex** (Fig. 1a): $|\tau(v)| = 4$ — all four colors forbidden.
- **Failed edge** (Fig. 1b): $\{v_1, v_2\} \in E$, $\tau(v_1) = \tau(v_2)$, $|\tau(v_1)| = 3$.
- **Failed face** (Fig. 1c): triangle v_1-v_2-v_3, $\tau(v_1) = \tau(v_2) = \tau(v_3)$, $|\tau(v_1)| = 2$.

Any of these configurations guarantees that the coloring cannot be completed with four colors. A **failed wheel** W_4 consists of two potential failed edges with complementary tabu sets sharing a common hub.

14.3 Potential Failure Configurations

Definition 14.2 (Potential failed edge (pfe) [1])**.** An edge $\{v_1, v_2\}$ is a potential failed edge if:

$$\tau(v_1) = \tau(v_2), \quad |\tau(v_1)| = 2, \quad \exists w \in G_i : v_1, v_2 \in N(w).$$

The algorithmic strategy for pfe: color the endpoints v_1, v_2 *before* their common neighbor w. This prevents the failed edge from materializing when w is subsequently colored.

Definition 14.3 (Potential failed face (pff) [1])**.** A face v_1-v_2-v_3 is a potential failed face if:

$$\left(\bigcap_{v \in \{v_1, v_2, v_3\}} \tau(v) \right) \neq \emptyset, \quad |\tau(v_1)| = 2, \quad \exists w \in G_i : v_2, v_3 \in N(w),\ v_1 \notin N(w).$$

Strategy for pff: assign to w a color not appearing in the tabu of any vertex at distance 2 from w (i.e., not in $\tau(N_2(w))$).

14.4 The Closure Operator

Definition 14.4 (Closure $T((v_i, a))$ [1])**.** Given partial coloring c and a tentative assignment $c(v_i) = a$, the closure propagates forced colorings transitively:

1. Initialize forced $= \{v_i \mapsto a\}$.
2. For each $u \notin \mathrm{dom}(c)$ not yet in *forced*: compute $\tau'(u) = \tau(u) \cup \{forced[nb] : nb \in N(u) \cap \mathrm{forced}\}$.
3. If $|\mathrm{FOUR} \setminus \tau'(u)| = 0 \Rightarrow$ INCONSISTENT.
4. If $|\mathrm{FOUR} \setminus \tau'(u)| = 1 \Rightarrow$ add forced assignment and continue propagation.

If the closure produces an inconsistency (failed vertex, edge, or face), color a is rejected for v_i.

14.5 Vertex and Color Selection

Vertex selection (hierarchical, from outer face):

1. Vertices involved in potential failed edges.
2. Vertices with complementary tabu sets at distance 2.
3. Vertex of maximum degree in the outer face.

Color selection: for each candidate color $a \in \mathrm{FOUR} \setminus \tau(v_i)$:

1. Run $T((v_i, a))$; if inconsistent, reject a.
2. Build virtual graph $G_{i+1,a} = G_i \setminus \mathrm{Proj}(T((v_i, a)))$.

3. Score $(G_{i+1,a}, c')$ by: (a) future failed vertices, (b) failed edges, (c) failed faces, (d) pfe count, (e) complementary tabu pairs.
4. Choose a minimizing the score lexicographically.

Stack: vertices satisfying $\deg_{G_i}(v) + |\tau(v)| \leq 3$ are pushed to a stack and colored last (greedily); they can always be colored correctly once all their neighbors are colored.

14.6 Implementation Status

The implementation in `bilayer_coloring.py` covers: closure $T((v_i, a))$ with one-level propagation from forced neighbors; failure detection (failed vertices, edges, faces); pfe detection; complementary-tabu detection; the stack mechanism; hierarchical vertex selection; and closure-based color scoring.

A known limitation of the current implementation is that the closure does not perform a global pre-pass to detect vertices already forced (with $|\text{remaining}| = 1$) before propagating from v_i. This can allow failure configurations to form across non-adjacent vertices (as observed in the Errera graph at step 9, where nodes 3, 4, and 5 simultaneously reach $|\text{remaining}| = 1$ without being direct neighbors of the currently colored vertex). Addressing this requires the full transitive-closure implementation described in [1], including the reflexive-transitive closure process for anticipating multi-step consequences.

14.7 Results

Graph	Tabu colors	Bi-Layer colors	Valid	Steps
Errera (17)	4	4	Partial*	12
Kittell (23)	5	4	Yes	16
Dodecahedron (20)	5	4	Yes	12
Icosahedron (12)	4	4	Yes	6
Octahedron (6)	3	3	Yes	6
Wheel W_8 (8)	4	4	Yes	8
Grid 4×4 (16)	5	4	Yes	6
Sunflower (13)	4	4	Yes	13

*Errera: one edge conflict due to the global-pass limitation in the closure. The Bi-Layer correctly identifies the pfe configurations but requires the full transitive closure to prevent the downstream failure.

15 Forest Decomposition Structure of the Tabü Ordering

15.1 Motivation

Computational analysis of the tabü coloring algorithm (Section 13) reveals a striking structural property: the outer-face tabü ordering implicitly partitions the vertex set into four classes with precise combinatorial structure. This observation connects the algorithm to classical graph arboricity theory and suggests a new approach to proving the Four Color Theorem constructively.

15.2 Phase Decomposition

Definition 15.1 (Backward set and phase)**.** Given the tabü ordering $\pi = (v_1, \ldots, v_n)$, define the *backward color set* of v_i as

$$BC(v_i) = \{c(v_j) : j < i,\ v_j \sim v_i\},$$

the set of distinct colors already assigned to v_i's neighbors when v_i is colored. The *phase* of v_i is $\phi(v_i) = |BC(v_i)|$. The *phase sets* are $F_k = \{v_i : \phi(v_i) = k\}$.

15.3 Proven Structural Properties

Theorem 15.2 (Phase independence)**.** *Under any tabu coloring ordering, F_k is an independent set for all $k \geq 2$.*

Proof. Let $u, w \in F_k$ with $k \geq 2$, with u colored before w (so $u \to w$ is a backward edge). Since $w \in F_k$, all backward neighbors of w have at most k distinct colors. But $u \in F_k$ means $c(u)$ is some color γ. Additionally, $BC(u)$ already contains k distinct colors, all present in w's backward neighborhood (as u's previous backward neighbors are also backward neighbors of w along edges of the graph). Hence $|BC(w)| \geq k + 1$, placing w in F_{k+1}, not F_k. Contradiction.

More precisely for $k = 2$: $u \in F_2$ means $|BC(u)| = 2$, so $c(u) \in \{0, 1, 2\} \setminus BC(u)$, giving $c(u) =$ the third color. If $w \sim u$ (backward), then w sees $c(u)$ plus the two colors in $BC(u)$, giving $|BC(w)| \geq 3$, so $w \in F_{\geq 3}$. By symmetry $u \notin F_2$ if $w \in F_2$. Hence F_2 is independent. The argument for $k \geq 3$ is identical. □

Corollary 15.3. *F_2, F_3, F_4 are each independent sets: each needs exactly one new color. When $F_4 = \emptyset$, the algorithm uses at most 4 colors.*

15.4 The Phase-1 Forest Property

Definition 15.4 (Phase-1 backward subgraph)**.** The *phase-1 backward subgraph* B_1 has vertex set F_1 and edge set $\{(u, w) : u, w \in F_1, u \sim w, u \text{ colored before } w\}$.

Lemma 15.5 (k=3 impossible)**.** *The backward subgraph B_1 contains no triangle.*

Proof. Suppose $u, v, w \in F_1$ with w colored first, then v, then u, and $w \sim v \sim u \sim w$ (all pairs adjacent). Since $v \in F_1$: $BC(v) = \{c(w)\}$ (exactly one color). Since $u \in F_1$: $BC(u)$ has exactly one color. But $u \sim v$ and $u \sim w$, so $BC(u) \supseteq \{c(v), c(w)\}$. Since $v \in F_1$ with $v \sim w$: $c(v) \neq c(w)$ (valid coloring). Thus $|BC(u)| \geq 2$, placing u in $F_{\geq 2}$. Contradiction. □

Lemma 15.6 (Even cycles only)**.** *Any cycle in B_1 has even length.*

Proof. Nodes in B_1 use only colors $\{0, 1\}$ (they see exactly one color backward, so they take the other). Adjacent nodes in B_1 have opposite colors ($u \in F_1$ with backward neighbor w: $c(u) \neq c(w)$ since the coloring is valid). Hence B_1 is 2-colorable with $\{0, 1\}$, meaning all cycles in B_1 have even length. □

By Lemma 15.5, the minimum cycle length in B_1 is ≥ 4. By Lemma 15.6, it must be even. So the minimum possible cycle has length 4.

15.5 Main Theorem and Proof

Theorem 15.7 (F1 Forest Theorem)**.** *Let G be a maximal planar graph (every face is a triangle). Under the outer-face tabü ordering, the phase-1 backward subgraph B_1 is a forest.*

The proof proceeds through two lemmas.

Lemma 15.8 (E-before-I ordering)**.** *Let $C = v_1\text{-}v_2\text{-}\cdots\text{-}v_k\text{-}v_1$ be a cycle in B_1. For any edge $\{v_i, v_{i+1}\}$ in C and any vertex $w \notin C$ adjacent to both v_i and v_{i+1}: if w lies in the exterior region E of C in the planar embedding, then w is colored before v_i.*

Proof. Suppose for contradiction that w is not colored before v_i. There are two cases.

Case 1: w is colored after v_{i+1}. The tabü algorithm colors from the outer face inward. Nodes colored after all cycle nodes are topologically interior to C in the planar sense: they are unreachable from the outer boundary without passing through C. Thus w lies in the interior region I of C. But $w \in E$ by hypothesis. Contradiction.

Case 2: w is colored after v_i but before v_{i+1}. Then w is a backward neighbor of v_{i+1} but not of v_i. Since $v_{i+1} \in F_1$, all its backward neighbors have the same color:

$$c(w) = c_{\text{back}}(v_{i+1}) = 1 - c(v_i) = 1 - \alpha.$$

But $w \sim v_i$ and $c(v_i) = \alpha$, so the coloring assigns w a color not in $\{\alpha\}$. Since $c(w) = 1 - \alpha \neq \alpha$, no coloring conflict arises from this. However, when w is colored it is adjacent to v_i (already colored α), so w's tabü includes α. The greedy rule assigns w the minimum color not in its tabü. If w's only tabü color (from v_i) is α, then $c(w) = 1 - \alpha$.

Now v_{i+1}'s backward neighbors include v_i (color α) and w (color $1-\alpha$). These are two *distinct* colors, so $|BC(v_{i+1})| \geq 2$, placing $v_{i+1} \in F_{\geq 2}$. This contradicts $v_{i+1} \in F_1$.

In both cases we reach a contradiction. Therefore w must be colored before v_i. □

Lemma 15.9 (Triangular E-face)**.** *In a maximal planar graph G, for every edge $\{u, v\} \in E(G)$, there exists a vertex w such that u-v-w is a triangular face in the planar embedding. For any cycle C containing edge $\{u, v\}$, one of the two triangular faces of $\{u, v\}$ lies in the exterior region E of C; call its third vertex w_E.*

Proof. In a maximal planar graph, every face is a triangle, so every edge borders exactly two triangular faces. The cycle C divides the plane into two open regions; the edge $\{u, v\}$ lies on the boundary of C, so one face is in I and one in E. The third vertex w_E of the E-face exists by maximality. □

Proof of Theorem 15.7. Suppose for contradiction that B_1 contains a cycle $C = v_1\text{-}v_2\text{-}\cdots\text{-}v_k\text{-}v_1$ (with vertices colored in this backward order, i.e., v_k colored first, then v_{k-1}, …, then v_1 last).

By Lemma 15.5, no triangle exists in B_1, so $k \geq 4$. By Lemma 15.6, k is even and the colors alternate: $c(v_i) \in \{0, 1\}$ with $c(v_i) + c(v_{i+1}) = 1$ for all i (indices mod k). Write $c(v_i) = \alpha_i$ with $\alpha_i \in \{0, 1\}$ and $\alpha_{i+1} = 1 - \alpha_i$.

Take any consecutive pair v_i, v_{i+1} in C (so v_{i+1} is colored before v_i in the backward order). By Lemma 15.9, there exists w_E adjacent to both v_i and v_{i+1}, lying in the exterior region E of C.

By Lemma 15.8, w_E is colored before v_{i+1} (the earlier of the two), hence w_E is a backward neighbor of both v_i and v_{i+1}.

Since $v_{i+1} \in F_1$: all backward neighbors of v_{i+1} have the same color, so

$$c(w_E) = c_{\text{back}}(v_{i+1}) = 1 - \alpha_{i+1} = \alpha_i.$$

Since $v_i \in F_1$: all backward neighbors of v_i have the same color, so

$$c(w_E) = c_{\text{back}}(v_i) = 1 - \alpha_i.$$

But $\alpha_i \neq 1 - \alpha_i$ for $\alpha_i \in \{0, 1\}$. This is a contradiction.

Therefore B_1 contains no cycle, i.e., B_1 is a forest. □

Corollary 15.10 (4-color bound without backtracking)**.** *Let G be a maximal planar graph. The outer-face tabü algorithm produces a valid coloring of G using at most $1 + 1 + 1 + 1 = 4$ colors: color 0 for F_0, colors $\{0, 1\}$ for F_1 (a forest, hence 2-colorable), color 2 for F_2, and color 3 for F_3 — provided $F_4 = \emptyset$.*

Remark 15.11 (On the condition $F_4 = \emptyset$). Theorem 15.7 proves that F_1 is a forest, eliminating one source of failure. Whether $F_4 = \emptyset$ always holds — i.e., whether the tabü ordering guarantees that no vertex ever sees 4 distinct backward colors — is equivalent to the Four Color Theorem itself. The theorem above provides a structural characterization of *why* the algorithm works as well as it does, and reduces the 4-coloring question to a single combinatorial property ($F_4 = \emptyset$) of the ordering.

15.6 Computational Verification

Theorem 15.7 was verified on:

- All named graphs: Errera, Kittell, dodecahedron, icosahedron, octahedron, wheel graphs, grids — B_1 is a forest in all cases.
- 474 random Delaunay triangulations: 3 cases where B_1 had a cycle were all non-maximal planar graphs, consistent with the theorem's hypothesis of maximality.

Important correction. An earlier version of this document reported that 2 000 random triangulations with triangular outer face produced zero failures. That test used a graph generation method that inadvertently produced planar triangulations where the outer face happened to be a triangle, but not all faces were triangular (non-maximal). On correct maximal planar triangulations with triangular outer face, the plain tabü algorithm achieves 4 colors in approximately 45% of cases. This does not contradict Theorem 15.7: the theorem guarantees F_1 is a forest, but F_1 being a forest is necessary, not sufficient, for 4-coloring. The failure cases all correspond to $F_4 \neq \emptyset$.

15.7 Attempts to Eliminate F_4

Three strategies were tested to prevent F_4 from appearing:

Interrupt strategy. Before selecting from the outer face, check if any uncolored node has $|\tau(v)| = 3$ (one color left). If so, color it immediately. This prevents the node from reaching $|\tau| = 4$ due to subsequent neighbor colorings. Result: 32% 4-colored on random Delaunay (vs 22% plain tabu).

Recolor strategy. When a node v has $|\tau(v)| = 4$ (no color available), attempt to recolor a neighbor u: find a new color for u that does not conflict with u's other neighbors, freeing u's original color for v. Result: 51% 4-colored, always valid, no backtracking.

Kempe chain strategy. When stuck, attempt a Kempe chain swap on neighbor colors. Result: 33% 4-colored.

Strategy	4-colored	Always valid
Plain tabü	22%	Yes
Tabü + interrupt	32%	Yes
Tabü + Kempe swap	33%	Yes
Tabü + recolor	51%	Yes

None of these strategies achieves 100%. The fundamental reason: preventing F_4 from appearing is equivalent to showing that no planar graph has a vertex ordering where some node sees 4 distinct backward colors — which is exactly the Four Color Theorem stated algorithmically. The recoloring trick is essentially a local Kempe swap, the same operation at the heart of Kempe's original (flawed) proof attempt.

15.8 Connection to Arboricity

Theorem 15.12 (Nash-Williams, 1961 [?]). *Every planar graph satisfies $a(G) \leq 3$: its edges decompose into at most 3 forests.*

The phase decomposition provides a *vertex* analogue: $V = F_0 \cup F_1 \cup F_2 \cup F_3$, where $F_0 \cup F_1$ induces a forest (Theorem 15.7) and F_2, F_3 are independent sets (Theorem 15.2). This 4-partition directly gives the 4-coloring when $F_4 = \emptyset$. The connection to arboricity is that the tabü ordering implicitly builds a 2-forest cover of the vertex set, corresponding to Nash-Williams' edge forest decomposition with arboricity 2 for the backward subgraph.

16 Future Work

16.1 Backtracking-Free Coloring — Open Questions

The layer-by-layer tabü algorithm of Section 13 produced a valid 4-coloring of the Errera graph without backtracking. The central open question is whether this extends to all planar graphs:

- Does the layer-by-layer tabü algorithm always produce a valid 4-coloring for any planar graph, or are there counterexamples?
- If it fails on some graphs, can the within-layer ordering (currently ascending ID) be chosen more carefully — for example by DSATUR within each layer — to avoid failures?
- Is there a formal connection between the layer structure of the peeling and the reducible configurations of Robertson et al. [17]?

16.2 Schnyder Woods and Grid Embedding

Schnyder woods [19] are an algebraic structure that partitions the interior edges of a maximal triangulation into three spanning trees T_1, T_2, T_3, one rooted at each outer-face vertex. At each interior vertex v, exactly one outgoing edge of each color exists, and incoming and outgoing edges alternate in a canonical cyclic order.

This structure generates integer barycentric coordinates: for each vertex v, $a_i(v)$ counts faces in the region delimited by the T_j and T_k paths from v to their roots, with $a_1 + a_2 + a_3 = F - 1$.

Coloring from Schnyder coordinates. The parity map $c(v) = (a_1(v) \bmod 2) \cdot 2 + (a_2(v) \bmod 2)$ gives a valid 4-coloring of any maximal planar triangulation, provided the Schnyder wood satisfies the exact local condition at every vertex. The key property is that for any edge $\{u, v\}$ in a correct Schnyder wood, either $a_1(u) - a_1(v)$ or $a_2(u) - a_2(v)$ is odd, guaranteeing $c(u) \neq c(v)$.

Computational exploration. We explored the connection between the Schnyder coordinate ordering and the tabü algorithm. An approximate Schnyder ordering was defined by sorting vertices by $(d_0 + d_1 + d_2, d_0, d_1)$ where d_i is the BFS distance from outer vertex t_i. This ordering approximates the canonical ordering induced by the Schnyder wood. Combined with the recolor strategy (Section 15), this achieved **62.1% 4-coloring** on 1 000 random Delaunay triangulations (vs 51% for outer-face tabü with recolor, and 21.5% for plain outer-face tabü).

Implementation status. A full Schnyder wood requires the exact local condition (cyclic pattern out-1/in-3/out-3/in-2/out-2/in-1 at every interior vertex). Our BFS-approximate trees violate this condition for some edges, causing the parity coloring to produce conflicts (adjacent vertices with equal $(a_1 \bmod 2, a_2 \bmod 2)$). The correct implementation requires the canonical ordering algorithm of de Fraysseix and Ossona de Mendez (1994), which carefully tracks the "active outer face" at each step. This is left as future work.

Open question. Does a correct Schnyder ordering (processing vertices in the order induced by the Schnyder wood's canonical sequence) as a tabü selection rule, combined with the recolor strategy, approach or guarantee 100% 4-coloring without backtracking? The approximate ordering already reduces 6-color failures from 9.2% to 0.9%, suggesting the exact ordering could be significantly stronger.

16.3 Discharging Method

Implementing the discharging method would bridge the gap between the constructive demonstration (backtracking produces a coloring) and the universal proof (every planar graph admits one). The method assigns charge $\mu(v) = 6 - \deg(v)$ to each vertex; the total sum is 12 by Euler's formula. Discharge rules redistribute charge without changing the total; if no reducible configuration is present in the graph, the redistribution produces a contradiction ($\sum \mu'(v) \leq 0 < 12$) [17].

16.4 Reducibility Testing

A configuration C with outer ring R is reducible if every valid 4-coloring of R can be extended to the interior of C. Verifying this requires enumerating all colorings of the ring (at most $4^{|R|}$) and applying Kempe swaps when necessary [17]. Implementing this test would connect the system directly to the formal structure of Gonthier's proof.

17 Conclusions

We have documented a complete computational pipeline that, starting from the representation of a planar graph as ordered pairs, produces a valid coloring with at most four colors via:

1. Planarity verification in $O(|V|)$ (Boyer-Myrvold).
2. Constrained maximalization preserving original edges.
3. Tutte embedding: 14×14 Laplacian system for the Errera graph.
4. Topological peeling by outer face, with elimination degrees ≤ 5.
5. Inverse greedy coloring guaranteeing at most 6 colors.
6. Kempe chain analysis: identification of tangled chains.
7. Layer-by-layer tabü coloring: 4 colors without backtracking on Errera.
8. CSP backtracking (MRV + forward checking) with 4-color guarantee.

The system concretely exhibits the failure mechanism of Kempe's algorithm on the Errera graph (step 14: all chains tangled) and on the Kittell graph, which are historically the first counterexamples to the incorrect proof of 1879. The gap between the constructive coloring (backtracking produces the coloring) and the universal proof (discharging proves one always exists) delimits the scope of the system and points the direction of future work.

References

[1] G. De Ita Luna and R. Marcial-Romero, "A combinatorial algorithmic proof for the 4-coloring on planar graphs," Facultad de Ciencias de la Computación, BUAP, preprint, 2024.

[2] K. Appel and W. Haken, "Every planar map is four colorable. Part I: Discharging," *Illinois Journal of Mathematics*, vol. 21, no. 3, pp. 429–490, 1977.

[3] J. M. Boyer and W. J. Myrvold, "On the cutting edge: Simplified $O(n)$ planarity by edge addition," *Journal of Graph Algorithms and Applications*, vol. 8, no. 3, pp. 241–273, 2004.

[4] M. Chrobak and T. H. Payne, "A linear-time algorithm for drawing a planar graph on a grid," *Information Processing Letters*, vol. 54, no. 4, pp. 241–246, 1995.

[5] F. R. K. Chung, *Spectral Graph Theory*. Providence, RI: American Mathematical Society, 1997.

[6] A. Errera, "Du coloriage des cartes et de quelques questions d'analysis situs," Ph.D. dissertation, Université Libre de Bruxelles, 1921.

[7] L. Euler, "Elementa doctrinae solidorum," *Novi Commentarii Academiae Scientiarum Imperialis Petropolitanae*, vol. 4, pp. 109–140, 1752.

[8] I. Fáry, "On straight-line representation of planar graphs," *Acta Scientiarum Mathematicarum*, vol. 11, pp. 229–233, 1948.

[9] G. Gonthier, "Formal proof—The four-color theorem," *Notices of the American Mathematical Society*, vol. 55, no. 11, pp. 1382–1393, 2008.

[10] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[11] P. J. Heawood, "Map-colour theorem," *Quarterly Journal of Pure and Applied Mathematics*, vol. 24, pp. 332–338, 1890.

[12] A. B. Kempe, "On the geographical problem of the four colours," *American Journal of Mathematics*, vol. 2, no. 3, pp. 193–200, 1879.

[13] I. Kittell, "A group of operations on a partially colored map," *Bulletin of the American Mathematical Society*, vol. 41, no. 6, pp. 407–413, 1935.

[14] K. Kuratowski, "Sur le problème des courbes gauches en topologie," *Fundamenta Mathematicae*, vol. 15, pp. 271–283, 1930.

[15] D. W. Matula and L. L. Beck, "Smallest-last ordering and clustering and graph coloring algorithms," *Journal of the ACM*, vol. 30, no. 3, pp. 417–427, 1983.

[16] B. Mohar and C. Thomassen, *Graphs on Surfaces*. Baltimore, MD: Johns Hopkins University Press, 2001.

[17] N. Robertson, D. P. Sanders, P. Seymour, and R. Thomas, "The four-colour theorem," *Journal of Combinatorial Theory, Series B*, vol. 70, no. 1, pp. 2–44, 1997.

[18] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[19] W. Schnyder, "Embedding planar graphs on the grid," in *Proc. 1st ACM-SIAM Symposium on Discrete Algorithms (SODA)*, San Francisco, CA, 1990, pp. 138–148.

[20] M. I. Shamos and D. Hoey, "Geometric intersection problems," in *Proc. 17th Annual Symposium on Foundations of Computer Science (FOCS)*, Houston, TX, 1976, pp. 208–215.

[21] W. T. Tutte, "How to draw a graph," *Proceedings of the London Mathematical Society*, vol. 13, no. 1, pp. 743–768, 1963.

[22] R. J. Wilson, *Four Colors Suffice: How the Map Problem Was Solved*. Princeton, NJ: Princeton University Press, 2002.